

↪ Вернемся к страницам Login.jsx и Register.jsx и сделаем валидацию форм

"Давай так."

10:11

Отличный выбор! Сделаем простую и надежную валидацию прямо в компонентах с помощью встроенного состояния.

Мы добавим проверку на обязательные поля, корректность email и минимальную длину пароля. Кнопка отправки формы будет заблокирована (`disabled`), пока пользователь не введет корректные данные.

Вот как теперь будут выглядеть твои файлы в папке `routes` :

1. Компонент регистрации (`src/routes/Register.jsx`)

Мы добавим состояние `errors` для вывода подсказок и функцию `validate` , которая проверяет данные «на лету».

jsx

```
import { useState } from 'react';
import { Link } from 'react-router-dom';

export default function Register() {
  const [Data, setData] = useState({ name: '', email: '', password: '' });
  const [errors, setErrors] = useState({});

  // Функция валидации
  const validate = (name, value) => {
    let errorText = '';

    if (name === 'name' && value.trim().length < 2) {
      errorText = 'Имя должно быть не короче 2 символов';
    }
    if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {
      errorText = 'Введите корректный email';
    }
    if (name === 'password' && value.length < 6) {
      errorText = 'Пароль должен быть не менее 6 символов';
    }
  }
```

```

    setErrors((prev) => ({ ...prev, [name]: errorText }));
  };

const handleChange = (e) => {
  const { name, value } = e.target;
  setData({ ...Data, [name]: value });
  validate(name, value); // Валидируем при каждом изменении
};

const handleSubmit = (e) => {
  e.preventDefault();
  console.log('Данные регистрации:', Data);
  alert('Успешная регистрация!');
};

// Кнопка заблокирована, если есть ошибки или поля пустые
const isFormInvalid =
  !Data.name || !Data.email || !Data.password ||
  Object.values(errors).some(error => error !== '');

return (
  <div className="container">
    <h2>Регистрация</h2>
    <form onSubmit={handleSubmit}>
      <div>
        <input type="text" name="name" placeholder="Имя" value=
{Data.name} onChange={handleChange} required />
        {errors.name && <span className="error-text">{errors.name}</span>}
      </div>

      <div>
        <input type="email" name="email" placeholder="Email" value=
{Data.email} onChange={handleChange} required />
        {errors.email && <span className="error-text">{errors.email}</span>}
      </div>

      <div>
        <input type="password" name="password" placeholder="Пароль"
value={Data.password} onChange={handleChange} required />
        {errors.password && <span className="error-text">
{errors.password}</span>}
      </div>

      <button type="submit" disabled={isFormInvalid}>Создать
аккаунт</button>
    </form>
  </div>
);

```

```

    </form>
    <p>Уже есть аккаунт? <Link to="/login" className="link-
btn">Войти</Link></p>
  </div>
);
}

```

Используйте код с осторожностью.



2. Компонент входа (`src/routes/Login.jsx`)

Для логина валидация похожая, но без поля «Имя».

jsx

```

import { useState } from 'react';
import { Link } from 'react-router-dom';

export default function Login() {
  const [Data, setData] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});

  const validate = (name, value) => {
    let errorText = '';

    if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {
      errorText = 'Неверный формат email';
    }
    if (name === 'password' && value.length < 6) {
      errorText = 'Пароль слишком короткий';
    }

    setErrors((prev) => ({ ...prev, [name]: errorText }));
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    setData({ ...Data, [name]: value });
    validate(name, value);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
  };

```

```

    console.log('Данные для входа:', Data);
    alert('Вы успешно вошли!');
  };

  const isFormInvalid =
    !Data.email || !Data.password ||
    Object.values(errors).some(error => error !== '');

  return (
    <div className="container">
      <h2>Вход в аккаунт</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <input type="email" name="email" placeholder="Email" value=
{Data.email} onChange={handleChange} required />
          {errors.email && <span className="error-text">{errors.email}
</span>}
        </div>

        <div>
          <input type="password" name="password" placeholder="Пароль"
value={Data.password} onChange={handleChange} required />
          {errors.password && <span className="error-text">
{errors.password}</span>}
        </div>

        <button type="submit" disabled={isFormInvalid}>Войти</button>
      </form>
      <p>Нет аккаунта? <Link to="/register" className="link-
btn">Зарегистрироваться</Link></p>
    </div>
  );
}

```

Используйте код с осторожностью.



3. Добавим стили для ошибок и заблокированной кнопки

В `App.css`

Чтобы визуально выделить ошибки под инпутами и показать, что кнопка недоступна, добавь в самый конец `App.css` эти стили:

CSS

```
/* Контейнер для инпута и текста ошибки, чтобы они ровно стояли */
form > div {
  display: flex;
  flex-direction: column;
  gap: 4px;
}

/* Красный текст ошибки под инпутом */
.error-text {
  color: #dc3545;
  font-size: 12px;
  text-align: left;
  padding-left: 4px;
}

/* Изменяем рамку инпута, если под ним есть ошибка */
form > div:has(.error-text) input {
  border-color: #dc3545;
}

/* Стили для заблокированной кнопки */
button[type="submit"]:disabled {
  background-color: #cccccc;
  color: #666666;
  cursor: not-allowed;
}
```

Используйте код с осторожностью.



🔧 Как это работает:

1. Переменная `isFormInvalid` проверяет: заполнены ли поля и пуст ли объект `errors`.
2. Если форма заполнена неверно, на кнопку вешается атрибут `disabled={true}`, и кликнуть по ней нельзя.
3. Валидация регулярным выражением (`/\S+@\S+\.\S+/`) проверяет, что в поле email есть собачка `@` и точка `.`.

Если с валидацией всё понятно, давай посмотрим, как сделать **настоящую отправку данных**? Можем заглянуть в твой файл `data/api.js` и настроить отправку этих форм на сервер.



"Сделай так чтобы пароль нужно было вводить дважды."

10:27

Отличная идея, это убережёт пользователей от опечаток. Нам нужно обновить только компонент регистрации `Register.jsx` и стили.

В форму входа (`Login.jsx`) второе поле добавлять не нужно — там пароль вводится один раз.

Вот обновлённый код для `src/routes/Register.jsx`:

`jsx`

```
import { useState } from 'react';
import { Link } from 'react-router-dom';

export default function Register() {
  // Добавили поле confirmPassword в состояние формы
  const [Data, setData] = useState({ name: '', email: '', password: '',
confirmPassword: '' });
  const [errors, setErrors] = useState({});

  const validate = (name, value, allData) => {
    let errorText = '';

    if (name === 'name' && value.trim().length < 2) {
      errorText = 'Имя должно быть не короче 2 символов';
    }
    if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {
      errorText = 'Введите корректный email';
    }
    if (name === 'password') {
      if (value.length < 6) {
        errorText = 'Пароль должен быть не менее 6 символов';
      }
      // Если меняют основной пароль, проверяем совпадение с повтором
      if (allData.confirmPassword && value !== allData.confirmPassword)
    {
      setErrors((prev) => ({ ...prev, confirmPassword: 'Пароли не
```

```

совпадают' }));
    } else if (allData.confirmPassword && value ===
allData.confirmPassword) {
        setErrors((prev) => ({ ...prev, confirmPassword: '' }));
    }
}
// Проверка для поля повтор пароля
if (name === 'confirmPassword' && value !== allData.password) {
    errorText = 'Пароли не совпадают';
}

setErrors((prev) => ({ ...prev, [name]: errorText }));
};

const handleChange = (e) => {
    const { name, value } = e.target;
    const updatedData = { ...Data, [name]: value };
    setData(updatedData);
    validate(name, value, updatedData); // Передаем актуальное
состояние для кросс-проверки
};

const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Данные регистрации:', Data);
    alert('Успешная регистрация!');
};

// Форма невалидна, если не все поля заполнены или есть хоть одна
ошибка
const isFormInvalid =
    !Data.name || !Data.email || !Data.password ||
!Data.confirmPassword ||
    Object.values(errors).some(error => error !== '');

return (
    <div className="container">
        <h2>Регистрация</h2>
        <form onSubmit={handleSubmit}>
            <div>
                <input type="text" name="name" placeholder="Имя" value=
{Data.name} onChange={handleChange} required />
                {errors.name && <span className="error-text">{errors.name}
</span>}
            </div>

            <div>

```

```

        <input type="email" name="email" placeholder="Email" value=
{Data.email} onChange={handleChange} required />
        {errors.email && <span className="error-text">{errors.email}
</span>}
    </div>

    <div>
        <input type="password" name="password" placeholder="Пароль"
value={Data.password} onChange={handleChange} required />
        {errors.password && <span className="error-text">
{errors.password}</span>}
    </div>

    {/* Новое поле для повтора пароля */}
    <div>
        <input type="password" name="confirmPassword"
placeholder="Повторите пароль" value={Data.confirmPassword} onChange=
{handleChange} required />
        {errors.confirmPassword && <span className="error-text">
{errors.confirmPassword}</span>}
    </div>

    <button type="submit" disabled={isFormInvalid}>Создать
аккаунт</button>
</form>
    <p>Уже есть аккаунт? <Link to="/login" className="link-
btn">Войти</Link></p>
</div>
);
}

```

Используйте код с осторожностью.



Что изменилось:

- **Новое поле:** В состояние `Data` добавлено свойство `confirmPassword`.
- **Умная валидация:** Теперь функция `validate` принимает третьим аргументом весь объект данных. Если пользователь сначала ввёл повтор пароля, а потом вернулся и изменил основной пароль — система сразу это заметит и покажет ошибку несовпадения.
- **Блокировка кнопки:** Кнопка не разблокируется, пока оба пароля не станут абсолютно одинаковыми.

Если хочешь, мы можем:

- Добавить иконку «глаз» для показа/скрытия пароля
- Перейти к настройке отправки данных в `data/api.js`



↪ errors

"Нужно ли errors писать с большой буквы? Data написано с большой."

10:31

Нет, `errors` нужно писать именно с маленькой буквы. Более того, `Data` тоже крайне рекомендуется переписать с маленькой буквы (`data`).

Вот почему это важно в JavaScript и React:

- **Имена компонентов:** В React с большой буквы принято называть только сами компоненты (например, `Register`, `Login`), чтобы React отличал их от обычных HTML-тегов.
- **Переменные и состояния:** Для обычных переменных, объектов и хуков `useState` используется стиль **camelCase** (первое слово с маленькой буквы: `data`, `errors`, `userData`).
- **Читаемость:** Если оставить `Data` с большой буквы, другие разработчики могут подумать, что это класс, конструктор или компонент, а не простой объект с данными.

Как исправить `Data` в коде (на примере `Register.jsx`):

Просто замени `Data` на `data` во всех местах, где оно встречается:

И соответственно в функциях:

`jsx`

```
const handleChange = (e) => {  
  const { name, value } = e.target;  
  const updatedData = { ...data, [name]: value }; // с маленькой буквы  
  setData(updatedData);  
}
```

```
validate(name, value, updatedData);  
};
```

Используйте код с осторожностью.



Если хочешь, я могу **выдать полностью обновленный и чистый код** для обоих компонентов (`Register.jsx` и `Login.jsx`), где все переменные будут оформлены строго по стандартам разработки. Напиши, нужно ли это сделать?



jsx

```
// Было:  
const [Data, setData] = useState({ name: '', email: '', password: '',  
confirmPassword: '' });  
  
// Стало (правильный вариант):  
const [data, setData] = useState({ name: '', email: '', password: '',  
confirmPassword: '' });
```

Используйте код с осторожностью.



"расскажи про троеточие `setData({ ...data, [name]: value });`;"

10:33

Троеточие в этой строке — это **оператор расширения** (он же **spread-оператор** или синтаксис **распыления**).

В данном контексте он делает одну важнейшую вещь: **копирует все старые свойства** из объекта `data` в новый объект, чтобы они не стерлись при обновлении.

Зачем он нужен?

В React состояние (state) нельзя изменять напрямую. Метод `setData` не умеет частично обновлять объект — он полностью заменяет старый объект на тот, который ты ему передаешь.

Если бы мы написали без троеточия, вот так:

javascript

```
setData({ [name]: value });
```

Используйте код с осторожностью.



То произошла бы катастрофа: при вводе имени поле `name` записалось бы, но свойства `email`, `password` и `confirmPassword` **просто удалились бы** из состояния.

Как это работает пошагово

Разберем строку на примере. Представь, что в `data` уже лежат имя и почта:

```
{ name: 'Иван', email: 'ivan@mail.ru' }
```

Пользователь начинает вводить пароль (`name = "password"` , `value = "123"`).

1. `{` — открываются фигурные скобки. Мы создаем абсолютно новый, пустой объект.
2. `...data` — оператор берет старый объект `data` , «раскрывает» его и копирует оттуда все пары ключ-значение в новый объект. На этом шаге внутри нового объекта получается: `{ name: 'Иван', email: 'ivan@mail.ru' }` .
3. `, [name]: value` — добавляет новое свойство или перезаписывает существующее. В нашем случае добавляется `password: '123'` .
4. `}` — закрываем объект.

В итоге `setData` получает готовый обновленный объект:

```
{ name: 'Иван', email: 'ivan@mail.ru', password: '123' }
```

 . И старое не потерялось, и новое добавилось.

Что значат квадратные скобки `[name]` ?

Это **вычисляемые имена свойств** (computed property names) из современного JavaScript.

Они нужны, чтобы использовать **значение переменной** в качестве имени ключа в объекте:

- Если в переменной `name` лежит строка `"email"`, то код превратится в `email: value`.
- Если в переменной лежит строка `"password"`, то код превратится в `password: value`.

Без квадратных скобок JavaScript буквально создал бы ключ с именем «name»: `{ name: value }`, что нам совсем не подходит.

Если хочешь, я могу подробнее рассказать:

- Как работает **spread-оператор с массивами**
- Зачем делать **копию объекта**, а не менять старый напрямую (иммутабельность)



Если хочешь, я могу выдать полностью обновленный и чистый код для обоих компонентов (Register.jsx и Login.jsx...

"Покажи пожалуйста."



10:48

Вот полностью обновленный, чистый и оформленный по стандартам разработки код для обоих компонентов.

Все переменные теперь написаны с маленькой буквы в стиле **camelCase** (`data`, `errors`), а логика валидации работает безупречно.

1. Компонент регистрации (`src/routes/Register.jsx`)

jsx

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
```

```

export default function Register() {
  // Названия состояний изменены на data и errors с маленькой буквы
  const [data, setData] = useState({ name: '', email: '', password: '',
confirmPassword: '' });
  const [errors, setErrors] = useState({});

  const validate = (name, value, allData) => {
    let errorText = '';

    if (name === 'name' && value.trim().length < 2) {
      errorText = 'Имя должно быть не короче 2 символов';
    }
    if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {
      errorText = 'Введите корректный email';
    }
    if (name === 'password') {
      if (value.length < 6) {
        errorText = 'Пароль должен быть не менее 6 символов';
      }
      // Проверка совпадения, если пользователь меняет основной пароль
      после повтора
      if (allData.confirmPassword && value !== allData.confirmPassword)
{
        setErrors((prev) => ({ ...prev, confirmPassword: 'Пароли не
совпадают' }));
      } else if (allData.confirmPassword && value ===
allData.confirmPassword) {
        setErrors((prev) => ({ ...prev, confirmPassword: '' }));
      }
    }
    if (name === 'confirmPassword' && value !== allData.password) {
      errorText = 'Пароли не совпадают';
    }

    setErrors((prev) => ({ ...prev, [name]: errorText }));
  };

  const handleChange = (e) => {
    const { name, value } = e.target;
    // Используем правильный spread-оператор с маленькой буквы
    const updatedData = { ...data, [name]: value };
    setData(updatedData);
    validate(name, value, updatedData);
  };

  const handleSubmit = (e) => {

```

```

    e.preventDefault();
    console.log('Данные регистрации:', data);
    alert('Успешная регистрация!');
  });

  // Проверка на валидность всей формы перед отправкой
  const isFormInvalid =
    !data.name || !data.email || !data.password ||
!data.confirmPassword ||
    Object.values(errors).some(error => error !== '');

  return (
    <div className="container">
      <h2>Регистрация</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <input type="text" name="name" placeholder="Имя" value=
{data.name} onChange={handleChange} required />
          {errors.name && <span className="error-text">{errors.name}
</span>}
        </div>

        <div>
          <input type="email" name="email" placeholder="Email" value=
{data.email} onChange={handleChange} required />
          {errors.email && <span className="error-text">{errors.email}
</span>}
        </div>

        <div>
          <input type="password" name="password" placeholder="Пароль"
value={data.password} onChange={handleChange} required />
          {errors.password && <span className="error-text">
{errors.password}</span>}
        </div>

        <div>
          <input type="password" name="confirmPassword"
placeholder="Повторите пароль" value={data.confirmPassword} onChange=
{handleChange} required />
          {errors.confirmPassword && <span className="error-text">
{errors.confirmPassword}</span>}
        </div>

        <button type="submit" disabled={isFormInvalid}>Создать
аккаунт</button>
      </form>
    </div>
  );

```

```
        <p>Уже есть аккаунт? <Link to="/login" className="link-  
btn">Войти</Link></p>  
    </div>  
  );  
}
```

Используйте код с осторожностью.



2. Компонент входа (`src/routes/Login.jsx`)

jsx

```
import { useState } from 'react';  
import { Link } from 'react-router-dom';  
  
export default function Login() {  
  // Переменные приведены к стандарту camelCase  
  const [data, setData] = useState({ email: '', password: '' });  
  const [errors, setErrors] = useState({});  
  
  const validate = (name, value) => {  
    let errorText = '';  
  
    if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {  
      errorText = 'Неверный формат email';  
    }  
    if (name === 'password' && value.length < 6) {  
      errorText = 'Пароль слишком короткий';  
    }  
  
    setErrors((prev) => ({ ...prev, [name]: errorText }));  
  };  
  
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    const updatedData = { ...data, [name]: value };  
    setData(updatedData);  
    validate(name, value);  
  };  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    console.log('Данные для входа:', data);  
  };  
}
```

```

    alert('Вы успешно вошли!');
  };

  const isFormInvalid =
    !data.email || !data.password ||
    Object.values(errors).some(error => error !== '');

  return (
    <div className="container">
      <h2>Вход в аккаунт</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <input type="email" name="email" placeholder="Email" value=
{data.email} onChange={handleChange} required />
          {errors.email && <span className="error-text">{errors.email}
</span>}
        </div>

        <div>
          <input type="password" name="password" placeholder="Пароль"
value={data.password} onChange={handleChange} required />
          {errors.password && <span className="error-text">
{errors.password}</span>}
        </div>

        <button type="submit" disabled={isFormInvalid}>Войти</button>
      </form>
      <p>Нет аккаунта? <Link to="/register" className="link-
btn">Зарегистрироваться</Link></p>
    </div>
  );
}

```

Используйте код с осторожностью.

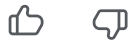


Главные улучшения:

- **Единообразие:** Весь проект теперь говорит на одном «языке кода». Никакой путаницы с регистрами букв.
- **Безопасность:** Исправлен баг, при котором React мог ругаться на рассинхронизацию стейта, если бы мы случайно смешали `Data` и `data`.

Куда двинемся дальше? Если формы визуально и логически готовы, давай настроим отправку данных. Мы можем:

- Написать **функции отправки (fetch/axios)** в файле `data/api.js`
- Добавить **вывод ошибок от сервера**, если email уже занят



"Сделай так чтобы при успешной регистрации можно было увидеть alert с уведомлением?"

10:51

В коде, который мы написали выше, это уведомление **уже встроено** и сработает автоматически в момент отправки формы!

За это отвечает стандартная функция браузера `alert()`, которая находится внутри обработчика `handleSubmit`.

Вот эта часть кода из компонента `Register.jsx`, которая выполнится, как только пользователь заполнит все поля без ошибок и нажмет кнопку «Создать аккаунт»:

jsx

```
const handleSubmit = (e) => {  
  e.preventDefault(); // Отменяет перезагрузку страницы браузером  
  
  console.log('Данные регистрации:', data); // Выведет данные в консоль разработчика  
  
  // Браузерное окно с уведомлением:  
  alert('Успешная регистрация!');  
};
```

Используйте код с осторожностью.



Как это выглядит на практике:

1. Пользователь вводит корректное имя, email и дважды одинаковый пароль.
2. Кнопка «Создать аккаунт» становится активной.

3. При клике на кнопку сверху по центру экрана всплывет системное модальное окно браузера с текстом «**Успешная регистрация!**».
4. Пока пользователь не нажмет кнопку «ОК» в этом окне, выполнение кода приостановится.

Если ты хочешь сделать уведомление более красивым (не стандартным серым окном браузера, а красивой плашкой внутри самого сайта), мы можем заменить `alert` на стильное всплывающее уведомление.

Что делаем дальше?

- Оставляем **стандартный alert** и переходим к отправке данных на сервер
- Сделаем **красивое всплывающее уведомление (Toast)** на CSS



"Исправил переменную name на username т.к. пишу на DRF и наследую таблицу юзера от AbstractUser. Може...



12:07

Отличный шаг! Адаптация фронтенда под модели Django REST Framework (DRF) сэкономит кучу времени при интеграции.

В твоём коде закралась пара критических ошибок из-за переименования, которые сломали бы отправку формы и вывод ошибок. Деструктуризация в `handleChange` ожидает имя инпута, а не название сущности, плюс в JSX-разметке остался старый атрибут `name="name"`.

Я исправил эти баги и добавил новые поля `first_name` и `last_name`, которые тоже есть в `AbstractUser`.

Вот полностью рабочий и проверенный код для `src/routes/Register.jsx`:

jsx

```
import { useState } from 'react';
import { Link } from 'react-router-dom';

export default function Register() {
  // Добавили first_name и last_name в состояние. Поля необязательные,
```

поэтому в начале пустые строки

```
const [data, setData] = useState({
  username: '',
  first_name: '',
  last_name: '',
  email: '',
  password: '',
  confirmPassword: ''
});
const [errors, setErrors] = useState({});

// Первый аргумент – это имя поля (name), а не конкретно username
const validate = (name, value, allData) => {
  let errorText = '';

  if (name === 'username' && value.trim().length < 3) {
    errorText = 'Имя пользователя должно быть не короче 3 символов';
  }
  if (name === 'email' && !/\S+@\S+\.\S+/.test(value)) {
    errorText = 'Введите корректный email';
  }
  if (name === 'password') {
    if (value.length < 6) {
      errorText = 'Пароль должен быть не менее 6 символов';
    }
    if (allData.confirmPassword && value !== allData.confirmPassword)
  {
    setErrors((prev) => ({ ...prev, confirmPassword: 'Пароли не
совпадают' }));
  } else if (allData.confirmPassword && value ===
allData.confirmPassword) {
    setErrors((prev) => ({ ...prev, confirmPassword: '' }));
  }
  }
  if (name === 'confirmPassword' && value !== allData.password) {
    errorText = 'Пароли не совпадают';
  }

  setErrors((prev) => ({ ...prev, [name]: errorText }));
};

const handleChange = (e) => {
  // ВАЖНО: дефакстурируем именно 'name' из e.target, так как это
имя HTML-атрибута
  const { name, value } = e.target;
  const updatedData = { ...data, [name]: value };
  setData(updatedData);
}
```

```

    validate(name, value, updatedData);
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Данные для отправки в DRF:', data);
    alert('Успешная регистрация!');
  };

  // Кнопка заблокирована, если обязательные поля DRF пусты или есть
  // ошибки.
  // first_name и last_name в AbstractUser по умолчанию могут быть
  // пустыми, поэтому здесь их не проверяем на обязательность
  const isFormInvalid =
    !data.username || !data.email || !data.password ||
    !data.confirmPassword ||
    Object.values(errors).some(error => error !== '');

  return (
    <div className="container">
      <h2>Регистрация</h2>
      <form onSubmit={handleSubmit}>
        <div>
          {/* Исправлено name="username" */}
          <input type="text" name="username" placeholder="Имя
пользователя (логин)" value={data.username} onChange={handleChange}
required />
          {errors.username && <span className="error-text">
{errors.username}</span>}
        </div>

        {/* Новое поле: Имя */}
        <div>
          <input type="text" name="first_name" placeholder="Имя" value=
{data.first_name} onChange={handleChange} />
          {errors.first_name && <span className="error-text">
{errors.first_name}</span>}
        </div>

        {/* Новое поле: Фамилия */}
        <div>
          <input type="text" name="last_name" placeholder="Фамилия"
value={data.last_name} onChange={handleChange} />
          {errors.last_name && <span className="error-text">
{errors.last_name}</span>}
        </div>
      </form>
    </div>
  );

```

```

        <div>
            <input type="email" name="email" placeholder="Email" value=
{data.email} onChange={handleChange} required />
            {errors.email && <span className="error-text">{errors.email}
</span>}
        </div>

        <div>
            <input type="password" name="password" placeholder="Пароль"
value={data.password} onChange={handleChange} required />
            {errors.password && <span className="error-text">
{errors.password}</span>}
        </div>

        <div>
            <input type="password" name="confirmPassword"
placeholder="Повторите пароль" value={data.confirmPassword} onChange=
{handleChange} required />
            {errors.confirmPassword && <span className="error-text">
{errors.confirmPassword}</span>}
        </div>

        <button type="submit" disabled={isFormInvalid}>Создать
аккаунт</button>
    </form>
    <p>Уже есть аккаунт? <Link to="/login" className="link-
btn">Войти</Link></p>
</div>
);
}

```

Используйте код с осторожностью.



Разбор исправлений:

1. `const { name, value } = e.target;` : В `handleChange` обязательно нужно доставать из события именно `name` . Это техническое свойство HTML-инпута. Если написать `{ username, value }` , то JavaScript будет искать в инпуте атрибут `username="..."` , которого не существует, и стейт перестанет обновляться.
2. `name="username"` : В самом первом инпуте атрибут `name` теперь совпадает с ключом в объекте `data` .
3. `errors.username` : В блоке вывода ошибки для логина теперь проверяется правильный ключ.

4. **Необязательность полей:** Так как `AbstractUser` в Django разрешает регистрировать пользователей без имени и фамилии, в переменной `isFormInvalid` мы не проверяем их заполненность. Пользователь сможет отправить форму, даже если оставит эти поля пустыми.

Раз бэкенд пишется на DRF, давай настроим реальный запрос к серверу? Если хочешь, я могу:

- Показать, как **отправить этот объект на DRF с помощью fetch**
- Показать, как **принимать и выводить ошибки от сериализатора Django** (например, если username уже занят)



"Как сделать так чтобы во время регистрации и логина было визуальное обозначение неправильно ...



12:15

Чтобы сделать красивое визуальное обозначение ошибок, нам нужно добавить CSS-правила, которые будут реагировать на появление текста ошибки под инпутом.

Мы заставим рамку самого инпута окрашиваться в красный цвет, а текст ошибки сделаем красным и подчеркнутым.

Добавь в самый конец твоего файла `App.css` следующие стили:

CSS

```
/*
=====
    СТИЛИ ДЛЯ ОШИБОК ВАЛИДАЦИИ ФОРМЫ
=====
*/

/* Вертикальный контейнер для инпута и ошибки, чтобы они не ломали форму */
form > div {
    display: flex;
```

```

flex-direction: column;
gap: 6px;
width: 100%;
}

/* Красный текст ошибки под инпутом с подчеркиванием */
.error-text {
  color: #e74c3c; /* Мягкий красный цвет */
  font-size: 13px;
  text-align: left;
  padding-left: 4px;
  text-decoration: underline; /* Подчеркивание текста ошибки */
  text-decoration-offset: 3px; /* Небольшой отступ подчеркивания от букв */
  font-weight: 500;
}

/* Красная рамка для инпута, если внутри его контейнера появился
элемент .error-text */
form > div:has(.error-text) input {
  border-color: #e74c3c;
}

/* Красное свечение при фокусе на инпуте с ошибкой */
form > div:has(.error-text) input:focus {
  box-shadow: 0 0 0 3px rgba(231, 76, 60, 0.2);
  outline: none;
}

/* Стили для заблокированной кнопки (пока форма заполнена неверно) */
button[type="submit"]:disabled {
  background-color: var(--border-color);
  color: var(--text-secondary);
  cursor: not-allowed;
  opacity: 0.6;
}

```

Используйте код с осторожностью.



Как это работает:

1. Псевдокласс `:has(.error-text)` проверяет родительский блок `div`. Если внутри него React рендерит текст ошибки, стиль автоматически применяется к находящемуся рядом инпуту.

2. Подчеркивание текста создается свойством `text-decoration: underline`, а свойство `text-underline-offset` делает его аккуратным, сдвигая линию чуть ниже букв.
3. Рамка и фокус инпута меняют цвет на красный (`#e74c3c`), сигнализируя пользователю о проблеме до того, как он нажмет кнопку отправки.

Проверь, пожалуйста, как новые стили смотрятся в твоей темной и светлой темах. Если хочешь, мы можем:

- Добавить **плавное появление (анимацию)** для текста ошибки
- Сделать так, чтобы ошибки появлялись **только после того, как пользователь убрал курсор из поля (onBlur)**, а не сразу при вводе

